

CSE5095: Types and Programming Languages

Homework 2: Type Safety

September 19, 2026

1 Booleans

Task 1. Write the inference rules for the dynamic semantics of Booleans and the if-else construct. You should have 2 new rules for the judgment $e \text{ val}$ and 3 new rules for the judgment $e \mapsto e'$.

Hint. The 2 value rules are the easy part: what are the only two Boolean literals, and by analogy with ValNum/ValString, what should their val rules look like (no premises)? For the 3 step rules on if e_1 then e_2 else e_3 : think about what has to happen *first* — the condition e_1 needs to be evaluated down to a value before you know which branch to take, so one rule should let you step e_1 while leaving the rest of the if-expression alone (compare to StepSearchAddLeft). The other 2 rules fire only once e_1 is already exactly true or exactly false — what should the whole if-expression step to in each case? (The assignment’s hint says exactly one of your three rules is a search rule — the other two should have no premises.)

Extra clue. Here are the shapes of the 5 rules with the interesting parts blanked out — you just need to fill in the _____’s:

$$\begin{array}{c} \frac{}{\text{_____ val}} \text{ (VALTRUE)} \qquad \frac{}{\text{_____ val}} \text{ (VALFALSE)} \\[1em] \frac{}{\text{if true then } e_2 \text{ else } e_3 \mapsto \text{_____}} \text{ (STEPIFTRUE)} \qquad \frac{}{\text{if false then } e_2 \text{ else } e_3 \mapsto \text{_____}} \text{ (STEPIFFALSE)} \\[1em] \frac{e_1 \mapsto e'_1}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \mapsto \text{_____}} \text{ (STEPSEARCHIF)} \end{array}$$

(For StepSearchIf: only the condition changes — the two branches e_2, e_3 should reappear on the right-hand side exactly as they were on the left.)

Your answer:

Task 2. Evaluate if $\bar{1} + \bar{2} = \bar{4}$ then “Fake news” else “True fact” fully, showing each step.

Hint. Work from the inside out. First the addition inside the condition needs to reduce (StepAdd) — but it’s nested inside an equality test, which is itself nested inside your if-condition, so you’ll need the appropriate search rules to “reach in” at each level (StepSearchEqLeft, then your if-search rule from Task 1). Once the condition becomes a closed numeral comparison, figure out whether StepEq or StepNotEq applies. Once the condition is fully a Boolean literal, one of your two “actual” if-step rules from Task 1 fires and picks a branch — that branch, if it’s already a string literal, is a value and you’re done.

Extra clue. There are exactly 3 steps. Here’s the skeleton with the intermediate expressions and rule names blanked out:

	if $\bar{1} + \bar{2} = \bar{4}$ then “Fake news” else “True fact”	
$\mapsto$		(which rule(s)?)
$\mapsto$		(which rule(s)?)
$\mapsto$		(which rule?)

Ask yourself at each line: is $\bar{1} + \bar{2}$ equal to $\bar{4}$ or not? That tells you whether StepEq or StepNotEq fires on the second step, which then tells you which of your two Task-1 branch rules fires on the third.

Your answer:

We also have the extended typing rules for equality testing and Booleans (given in the assignment):

$$\begin{array}{c}
 \frac{e_1 : \text{int} \quad e_2 : \text{int}}{e_1 = e_2 : \text{bool}} \text{ (TYPEEQ)} \qquad \frac{}{\text{true} : \text{bool}} \text{ (TYPETRUE)} \qquad \frac{}{\text{false} : \text{bool}} \text{ (TYPEFALSE)} \\
 \\
 \frac{e_1 : \text{bool} \quad e_2 : \tau \quad e_3 : \tau}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \tau} \text{ (TYPEIF)}
 \end{array}$$

Task 3. Give a derivation using the extended typing rules for if $\bar{1} + \bar{2} = \bar{4}$ then “Fake news” else “True fact” : string.

Hint. Build the tree bottom-up, the same way the class notes built the derivation for $\bar{0} + ((\bar{1} + \bar{2}) + \bar{3}) : \text{int}$. Type the two numerals under the addition (TypeNum), combine with TypeAdd, then bring in $\bar{4}$ and combine with TypeEq to get the condition typed **bool**. Separately, type each string branch with TypeString. Finally combine all three pieces with TypeIf — remember both branches must be assigned the *same* τ (here, **string**) for TypeIf to apply. Abbreviate rule names (e.g. TN, TA, TEq, TS, TIf) so the tree fits on the page; \small or \scriptsize around the whole derivation can help too.

Extra clue. Here is the shape of the tree with only the leaves’ rule names filled in — you fill in every conclusion (the “... : τ ” under each rule) and the two internal rule names:

$$\begin{array}{c}
 \frac{}{\bar{1} : \underline{\hspace{1cm}}} \text{ (TN)} \qquad \frac{}{\bar{2} : \underline{\hspace{1cm}}} \text{ (TN)} \\
 \hline
 \frac{}{\hspace{1.5cm}} (?) \qquad \frac{}{\bar{4} : \underline{\hspace{1cm}}} \text{ (TN)} \\
 \hline
 \frac{}{\hspace{3cm}} (?)
 \end{array}$$

which then becomes the *first* premise of the final TypeIf step (the other two premises are the two TypeString leaves for the branches).

Your answer:

Task 4. Prove the cases of the Preservation theorem for the new rules you added in Task 1.

Hint. Recall **Theorem (Preservation)**. If $e : \tau$ and $e \mapsto e'$, then $e' : \tau$. You're doing induction on the derivation of $e \mapsto e'$, with one case per rule from Task 1. For the two non-search rules (where the if-expression steps directly to a branch), use *inversion* on TypeIf to read off what you already know about that branch's type — no induction hypothesis needed there. For the search-rule case, you'll need to apply your induction hypothesis to the sub-step of the condition, then re-apply TypeIf to reassemble the result. This should closely mirror the class's proof of the StepSearchAddLeft case for the original Preservation theorem.

Extra clue. Fill in the blanks in this proof skeleton (compare directly against the class's StepAdd and StepSearchAddLeft cases in the lecture notes):

- **Case StepIfTrue.** Then $e = \underline{\hspace{2cm}}$ and $e' = \underline{\hspace{2cm}}$. By inversion on $\underline{\hspace{2cm}}$, we get $\underline{\hspace{2cm}} : \tau$ and $\underline{\hspace{2cm}} : \tau$. In particular, $e' : \tau$.
- **Case StepIfFalse.** (Symmetric to the above — which branch do you keep this time?)
- **Case StepSearchIf.** Then $e = \underline{\hspace{2cm}}$, $e' = \underline{\hspace{2cm}}$, with $e_1 \mapsto e'_1$. By inversion on TypeIf: $e_1 : \underline{\hspace{2cm}}$, $e_2 : \tau$, $e_3 : \tau$. By the induction hypothesis applied to $e_1 \mapsto e'_1$, conclude $e'_1 : \underline{\hspace{2cm}}$. By $\underline{\hspace{2cm}}$ again (this time with e'_1 in place of e_1), $e' : \tau$.

Your answer:

Task 5. State (you do not have to prove it) the new case of the Canonical Forms lemma for Booleans.

Hint. Look at the existing Canonical Forms statement for `int` and `string` (“if e `val` and $e : \tau$, then e looks like ...”). What are the *only* two values that could possibly have type `bool`, given the value rules from Task 1?

Extra clue. Complete the sentence: “If e `val` and $e : \text{bool}$, then $e =$ _____ or $e =$ _____.” (One quick way to convince yourself this is right: which value rule from Task 1 would you use to justify each disjunct?)

Your answer:

Task 6. Prove the cases of the Progress theorem for the new rules *TypeTrue*, *TypeFalse* and *TypeIf*. You may (and should) use the new case of Canonical Forms from Task 5.

Hint. Recall **Theorem (Progress)**. If $e : \tau$, then e `val` or there exists e' with $e \mapsto e'$. Induction is on the derivation of $e : \tau$. *TypeTrue*/*TypeFalse* should be one-line cases (compare to the *TypeNum*/*TypeString* cases in the notes). For *TypeIf*, apply the induction hypothesis to the sub-derivation of the *condition*’s typing judgment to get two branches (condition is a value, or condition steps); in the “condition is a value” branch, use your Task 5 Canonical Forms lemma to pin down exactly which of the two Boolean literals it must be, and then pick the corresponding rule from Task 1.

Extra clue. Fill in the blanks (this should look almost identical in structure to the class’s *TypeAdd* case, just one level instead of two nested inductions):

- **Case TypeTrue.** Then $e =$ _____. By _____, e `val`.
- **Case TypeFalse.** (Same idea, other literal, other rule.)
- **Case TypeIf.** Then $e = \text{if } e_1 \text{ then } e_2 \text{ else } e_3$ with $e_1 : \text{_____}$, $e_2 : \tau$, $e_3 : \tau$. By the induction hypothesis on $e_1 : \text{_____}$: either e_1 `val` or $e_1 \mapsto e'_1$.
 - If e_1 `val`: by _____ (Task 5), $e_1 = \text{true}$ or $e_1 = \text{false}$. In the first case, use rule _____ to show $e \mapsto$ _____; in the second, use rule _____ to show $e \mapsto$ _____.
 - If $e_1 \mapsto e'_1$: use rule _____ to show $e \mapsto$ _____.

Your answer:

2 Division by zero

Task 7. Fill in the blanks in the typing rules for EZ.

Hint. For each operation, ask: *can* the result be zero, given what you know (or don't know) about the operands? For $+$ and $-$: if both operands are only known to be “maybe zero,” can the sum/difference still come out to zero? (Try small examples, and remember integers can be negative.) For $*$: the two given rules already show the multiplication case is split depending on whether both factors are known-nonzero or not — think about *why* two nonzero integers multiplied together can never be zero. For $/$: what does the operational rule StepDiv require of the second operand in order to not get stuck? That tells you the required type of e_2 in TypeDiv. Then ask: even when e_1 and e_2 are both guaranteed nonzero, can integer division still produce a zero result? (Think about truncating division, e.g. small numerator, larger denominator.) That tells you the result type.

$$\begin{array}{c}
 \frac{}{\bar{n} : \text{mz}} \text{ (TypeNum)} \qquad \frac{n \neq 0}{\bar{n} : \text{nz}} \text{ (TypeNumNZ)} \\
 \\
 \frac{e_1 : \text{mz} \quad e_2 : \text{mz}}{e_1 + e_2 : ?} \text{ (TypeAdd)} \qquad \frac{e_1 : \text{mz} \quad e_2 : \text{mz}}{e_1 - e_2 : ?} \text{ (TypeSub)} \\
 \\
 \frac{e_1 : \text{nz} \quad e_2 : \text{nz}}{e_1 * e_2 : \text{nz}} \text{ (TypeMulNZ)} \qquad \frac{e_1 : \text{mz} \quad e_2 : \text{mz}}{e_1 * e_2 : \text{mz}} \text{ (TypeMulMZ)} \\
 \\
 \frac{e_1 : \text{mz} \quad e_2 : ?}{e_1 / e_2 : ?} \text{ (TypeDiv)}
 \end{array}$$

Extra clue. Work through these small numeric checks by hand — each one tells you directly whether a given blank must be `mz` or can safely be `nz`:

- Is there a choice of nonzero-or-zero integers n_1, n_2 with $n_1 + n_2 = 0$? (Try $n_1 = 3$.) What does that tell you about the blank in TypeAdd?
- Is there a choice of n_1, n_2 with $n_1 - n_2 = 0$? What does that tell you about the blank in TypeSub?
- Look back at the operational rule StepDiv: what side condition does it place on the second operand n_2 ? That's exactly the type you need for e_2 in TypeDiv (first blank).
- Using integer (truncating) division, is $\bar{1}/\bar{2}$ equal to $\bar{0}$? If both operands of $/$ could be nonzero and the result can *still* be zero, what must the result type (second blank in TypeDiv) be?

Your answer:

Task 8. Prove the Progress and Preservation theorems for EZ (cases TypeNum, TypeNumNZ, TypeDiv for Progress; StepAdd, StepSub, StepDiv for Preservation).

Hint. These should closely mirror the class's proofs of Progress and Preservation for the E language (TypeAdd/StepAdd cases), just swapping in your Task 7 rules and the EZ Canonical Forms lemma (given in the assignment) in place of the int/string versions. For the Progress/TypeDiv case in particular: apply your induction hypothesis to *both* e_1 and e_2 (nested, the way the class's TypeAdd case does), use Canonical Forms to turn “value” into “some numeral,” and remember that StepDiv has a side condition ($n_2 \neq 0$) that your typing hypothesis on e_2 should give you for free. For Preservation, each case is short: invert on the relevant typing rule to pin down τ , then use TypeNum to retype the result.

Extra clue. Progress, fill in the blanks (case TypeDiv only — TypeNum/TypeNumNZ are one-liners like TypeTrue above):

- **Case TypeDiv.** Then $e = e_1/e_2$, $\tau = \underline{\hspace{1cm}}$, with $e_1 : \text{mz}$ and $e_2 : \underline{\hspace{1cm}}$. By IH on e_1 : $e_1 \text{ val}$ or $e_1 \mapsto e'_1$.
 - If $e_1 \text{ val}$: by Canonical Forms, $e_1 = \overline{n_1}$. By IH on e_2 : $e_2 \text{ val}$ or $e_2 \mapsto e'_2$.
 - * If $e_2 \text{ val}$: by Canonical Forms (the **nz** case!), $e_2 = \overline{n_2}$ with $\underline{\hspace{1cm}}$. So by rule $\underline{\hspace{1cm}}$, $e \mapsto \underline{\hspace{1cm}}$.
 - * If $e_2 \mapsto e'_2$: by rule $\underline{\hspace{1cm}}$, $e \mapsto \underline{\hspace{1cm}}$.
 - If $e_1 \mapsto e'_1$: by rule $\underline{\hspace{1cm}}$, $e \mapsto \underline{\hspace{1cm}}$.

Preservation, fill in the blanks (all three cases follow the same 2-line pattern):

- **Case StepAdd.** $e = \overline{n_1} + \overline{n_2}$, $e' = \overline{n_1 + n_2}$. By inversion on $\underline{\hspace{1cm}}$, $\tau = \underline{\hspace{1cm}}$. By $\underline{\hspace{1cm}}$, $e' : \tau$.
- **Case StepSub.** (Same pattern, different typing rule to invert on.)
- **Case StepDiv.** (Same pattern again — which rule do you invert on this time?)

Your answer:

Task 9 (Bonus). *Prove the StepMul case for Preservation. **Hint (from the assignment):** if you do inversion on the typing rules, there will actually be two rules you need to consider in your inversion, and thus two cases.*

Hint. A product $e_1 * e_2$ could have been typed by *either* TypeMulNZ or TypeMulMZ — inversion on the typing derivation doesn't hand you a unique rule the way it did for StepAdd, so you'll genuinely need two sub-cases. In the TypeMulNZ sub-case, invert once more (on TypeNumNZ) to learn both numerals are nonzero, then think about why a product of two nonzero integers can't be zero. In the TypeMulMZ sub-case, there's much less to check.

Extra clue. Fill in the blanks:

- **Case StepMul.** $e = \overline{n_1} * \overline{n_2}$, $e' = \overline{n_1 n_2}$. Inversion on the derivation of $e : \tau$ gives two sub-cases.
 - **Via TypeMulNZ.** $\tau = \underline{\hspace{1cm}}$, and $\overline{n_1} : \text{nz}$, $\overline{n_2} : \text{nz}$. Inverting *again* on $\underline{\hspace{1cm}}$, we learn $n_1 \neq \underline{\hspace{1cm}}$ and $n_2 \neq \underline{\hspace{1cm}}$. A fact from basic arithmetic you're allowed to use without proof: the product of two nonzero integers is $\underline{\hspace{1cm}}$. So by rule $\underline{\hspace{1cm}}$, $e' : \underline{\hspace{1cm}} = \tau$.
 - **Via TypeMulMZ.** $\tau = \underline{\hspace{1cm}}$ (no constraint on n_1, n_2). By rule $\underline{\hspace{1cm}}$, $e' : \underline{\hspace{1cm}} = \tau$.

Your answer:

Task 10. Give a syntactically valid expression in EZ that does not perform a division by zero, but is still not well-typed by the above typing rules. Informally suggest a change, addition, refinement, etc., to the type system that could make your expression well-typed (but would still rule out expressions that divide by zero).

Hint. Look for a gap between what's *true at runtime* and what the type system can *prove*. In particular, notice that TypeAdd and TypeSub (as you filled them in for Task 7) only ever conclude `mz` — there's no rule that lets a sum or difference be typed `nz`, no matter what its operands are. Now think of a division whose divisor is written as a sum or difference of two numerals that you, doing the arithmetic yourself, know is nonzero. For the refinement part: you don't need to design a whole new system, just gesture at what extra information the type system would need to track (e.g., sign, or a way to reason about specific known numeral values) to let that particular divisor be typed `nz`, while thinking about why your fix wouldn't accidentally let an actual division-by-zero back in.

Extra clue. Concretely: pick any two small numerals n_2, n_3 whose sum (or difference) you can verify by hand is nonzero, and any numeral n_1 at all, and consider the expression $\overline{n_1}/(\overline{n_2} + \overline{n_3})$ (or with $-$ instead of $+$). Walk through why TypeDiv can never accept this: what type does TypeAdd force on $\overline{n_2} + \overline{n_3}$, and how does that compare to what TypeDiv requires of its second argument? For the refinement, it can help to ask: what's the *smallest* piece of information beyond “maybe zero” you could attach to a numeral (or an expression) that would let you conclude a sum is nonzero — and would every numeral automatically have that information, or only some?

Your answer:

3 One more wrap-up question

Task 11. How long (approximately) did you spend on this homework, in total minutes/hours of actual working time? Your honest feedback will help me with future homeworks.

Your answer: